Transformers

Greg Strabel

March 23, 2025

Contents

1 Summary

This document provides a brief overview of many of the most important papers on transformers, the neural network architecture that has produced significant advances in NLP in recent years.

2 Preliminaries

2.1 Matrix Multiplication

Given two matrices $X \in \mathbb{R}^{n \times m}$ and $Y \in \mathbb{R}^{m \times k}$

$$(XY)_{ij} = \sum_{l=1}^{m} X_{il} Y_{lj} = X_{i\cdot} Y_{\cdot j} \tag{1}$$

Therefore

$$(XY)_{\cdot j} = \sum_{l=1}^{m} Y_{lj} X_{\cdot l} \tag{2}$$

so that the columns of XY are linear combinations of the columns of X and

$$(XY)_{i\cdot} = \sum_{l=1}^{m} X_{il} Y_{l\cdot} \tag{3}$$

so that the rows of XY are linear combinations of the rows of Y.

2.2 Softmax

Definition 2.1 (Softmax)**.** The softmax function $\sigma : \mathbb{R}^K \to \mathbb{R}^K$ is

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{4}$$

2.3 Softplus

Definition 2.2 (Softplus)**.** The Softplus function is a smooth approximation to the ReLU function and constrains the output to always be positive. It is defined as:

$$Softplus(x) = \frac{1}{\beta} \cdot \log\left(1 + \exp\left(\beta \cdot x\right)\right) \tag{5}$$

2.4 Evaluating Text Quality

Definition 2.3 (n-grams)**.** The n-grams of a string $y = y_1 \ldots y_K$ are

$$G_n(y) = \{y_1 \ldots y_n, y_2 \ldots y_{n+1}, \ldots, y_{K-n+1} \ldots y_K\} \tag{6}$$

Note that $G_n(y)$ is a set, not a multiset, so the elements of $G_n(y)$ are distinct.

Definition 2.4 (Substring Count)**.** Given two strings, $s = s_1...s_M$ and $y = y_1...y_K$ the substring count of substring s in y is

$$C(s, y) = |\{i \in \mathbb{N} \mid i > 0,\ i + M \leq K,\ y_i...y_{i+M} = s\}| \tag{7}$$

Definition 2.5 (Modified n-gram precision)**.** Given a candidate corpus $\hat{S} = (\hat{y}^{(1)}, \hat{y}^{(2)}, ..., \hat{y}^{(M)})$ and corresponding reference corpi $S = (S_1, S_2, ..., S_M)$ where $S_i = (y^{(i,1)}, y^{(i,2)}, ..., y^{(i,N_i)})$, modified n-gram precision is defined as

$$p_n(\hat{S}, S) = \frac{\sum_{i=1}^{M} \sum_{s \in G_n(\hat{y}^{(i)})} \min(C(s, \hat{y}^{(i)}), \max_{y \in S_i} C(s, y))}{\sum_{i=1}^{M} \sum_{s \in G_n(\hat{y}^{(i)})} C(s, \hat{y}^{(i)})} \tag{8}$$

Definition 2.6 (Brevity penalty)**.** Given a candidate corpus $\hat{S} = (\hat{y}^{(1)}, \hat{y}^{(2)}, ..., \hat{y}^{(M)})$ and corresponding reference corpi $S = (S_1, S_2, ..., S_M)$ where $S_i = (y^{(i,1)}, y^{(i,2)}, ..., y^{(i,N_i)})$, the brevity penalty is defined as

$$BP(\hat{S}, S) = e^{-(r/c-1)^+} \tag{9}$$

where

$$c = \sum_{i=1}^{M} |\hat{y}^{(i)}| \tag{10}$$

is the length of the candidate corpus and

$$r = \sum_{i=1}^{M} |\text{argmin}_{y \in S_i} \left(||y| - |\hat{y}^{(i)}|| \right) | \tag{11}$$

is the effective reference corpus length.

Definition 2.7 (Bilingual Evaluation Understudy (BLEU))**.** Given a candidate corpus $\hat{S} = (\hat{y}^{(1)}, \hat{y}^{(2)}, ..., \hat{y}^{(M)})$, a corresponding reference corpi $S = (S_1, S_2, ..., S_M)$ where $S_i = (y^{(i,1)}, y^{(i,2)}, ..., y^{(i,N_i)})$, and $w \in \{[0, 1]^\infty : \sum_{i=1}^{\infty} w_i = 1\}$, the BLEU score is defined as:

$$BLEU_w(\hat{S}, S) = BP(\hat{S}, S) \cdot \exp\left(\sum_{n=1}^{\infty} w_n \ln p_n(\hat{S}, S)\right) \tag{12}$$

Definition 2.8 (ROUGE-n)**.** Given a candidate sentence $\hat{y}$ and a corresponding corpus of reference sentences $S = \{y^{(1)}, y^{(2)}, ..., y^{(N)}\}$, $ROUGE - n$ is defined as

$$ROUGE - n(\hat{y}, S) = \sum_{i=1}^{N} \frac{\sum_{s \in G_n(y^{(i)})} \min(C(s, \hat{y}), C(s, y))}{\sum_{s \in G_n(y^{(i)})} C(s, y)} \tag{13}$$

Note that the definition of ROUGE metrics varies across sources. The definitions above align with [Sch17].

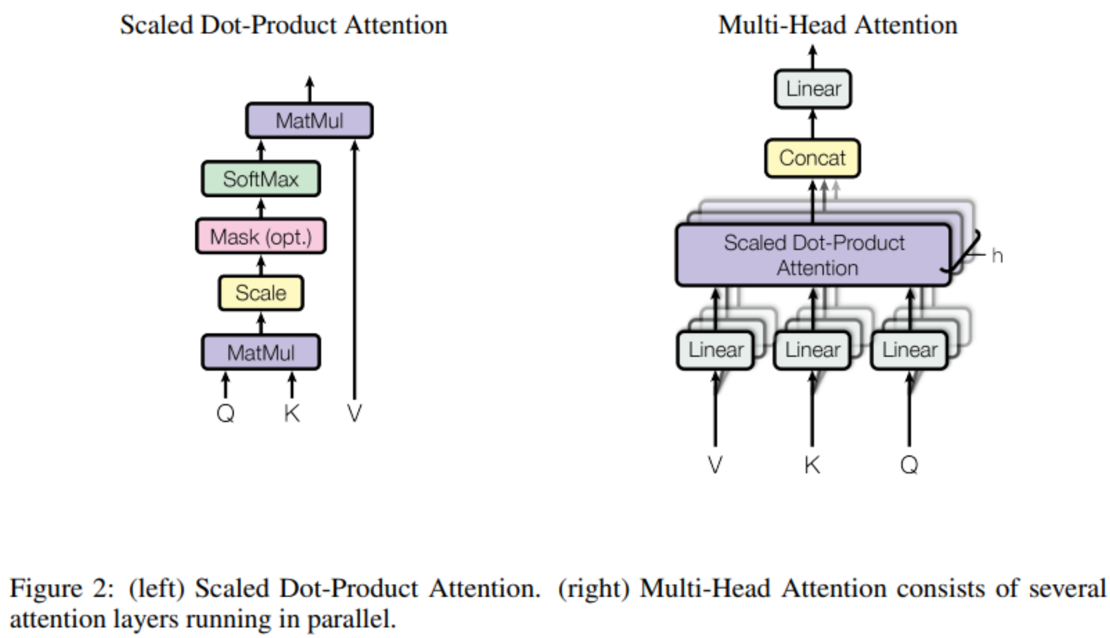


Figure 1: Attention Mechanism [VSP+17]

3 Attention

3.1 Dot-Product Attention

Definition 3.1 (Dot-Product Attention)**.** Given $Q \in \mathbb{R}^{d_l \times d_k}$, $K \in \mathbb{R}^{d_s \times d_k}$ and $V \in \mathbb{R}^{d_s \times d_v}$

$$\text{Attention}\,(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \in \mathbb{R}^{d_l \times d_v} \tag{14}$$

3.2 Multihead Attention

Given input matrices $X_Q \in \mathbb{R}^{d_l \times d_g}$, $X_K \in \mathbb{R}^{d_s \times d_w}$ and $X_V \in \mathbb{R}^{d_s \times d_u}$ and weight matrices

$$\begin{aligned} &\{W_Q^i \in \mathbb{R}^{d_g \times d_k}\}_{i=1}^h \\ &\{W_K^i \in \mathbb{R}^{d_w \times d_k}\}_{i=1}^h \qquad (15) \\ &\{W_V^i \in \mathbb{R}^{d_u \times d_v}\}_{i=1}^h \\ &W_O \in \mathbb{R}^{hd_v \times d_o} \qquad (16) \end{aligned}$$

we define

$$
\begin{aligned}
Q^i &= X_Q W_Q^i \in \mathbb{R}^{d_l \times d_k} \\
K^i &= X_K W_K^i \in \mathbb{R}^{d_s \times d_k} \\
V^i &= X_V W_V^i \in \mathbb{R}^{d_s \times d_v}
\end{aligned}
\tag{17}
$$

$$
A^i = \text{Attention}\left(Q^i, K^i, V^i\right) \in \mathbb{R}^{d_l \times d_v} \tag{18}
$$

$$
\text{Multihead}\left(X_Q, X_K, X_V\right) = \text{concat}\left(A_1, ..., A_h\right) W_O \in \mathbb{R}^{d_l \times d_o} \tag{19}
$$

3.3 Multihead Self-Attention

In Multihead Self-Attention, $X_Q = X_K = X_V$ so that $d_{model} \coloneqq d_g = d_w = d_u$, $L \coloneqq d_l = d_s$ and $\text{Multihead}\left(X_Q, X_K, X_V\right) \in \mathbb{R}^{L \times d_o}$. If we also have $d_o = d_{model}$ then we get the Multihead Self-Attention described in Attention Is All You Need ([VSP+17]).

3.4 Transformer Block

A single transformer block (see Figure 2) takes an input tensor X_0 and then:

1. Applies a multi-head attention layer to X_0 to produce X_1
2. Adds X_0 and X_1 and applies normalization to produce X_2
3. Applies a fully connected feed forward layer to X_2 to produce X_3
4. Adds X_2 and X_3 and applies normalization to produce the final output X_4

4 Attention is All You Need [VSP+17]

Multi-head self-attention was introduced in the paper Attention is All You Need [VSP+17], which used the architecture in Figure 3 for a translation task.

Inputs to the model are Byte Pair Encoded (BPE, section 18) tokens, which are mapped to a learned embedding space. In order for the model to make use of the order of the input sequence, information on sequence order is injected by adding in sine and cosine function of different frequencies:

$$
\begin{aligned}
PE_{(pos,2i)} &= \sin(pos/10000^{2i/d_{model}}) \\
PE_{(pos,2i+1)} &= \cos(pos/10000^{2i/d_{model}})
\end{aligned}
\tag{20}
$$

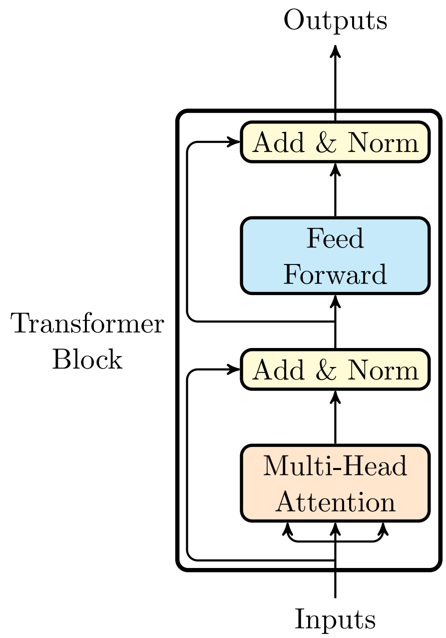


Figure 2: Single Transformer Block

5 Improving Language Understanding by Generative Pre-Training

[RNSS18] is the OpenAI article that introduced GPT (Generative Pre-Training). This framework uses unsupervised pre-training of a multi-layer transformer decoder with a masked language model objective followed by fine-tuning for specific tasks.

Unsupervised pre-training involves taking a corpus of tokens $\mathcal{U} = \{u_1, u_2, ..., u_n\}$ and training a model to maximize the likelihood objective:

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, ..., u_{i-1}; \Theta) \tag{21}$$

where k is the size of the context window, P is the conditional probability of the next token and Θ are the parameters of the model.

[RNSS18] use a multi-layer transformer decoder for their model; a stack of multiple transformer blocks:

$$\begin{aligned} & h_0 = UW_e + W_p \\ & h_l = \texttt{transformer\_decoder}(h_{l-1}) \quad \forall i \in [1, ..., n] \\ & P(u) = \texttt{softmax}(h_n W_e') \end{aligned} \tag{22}$$

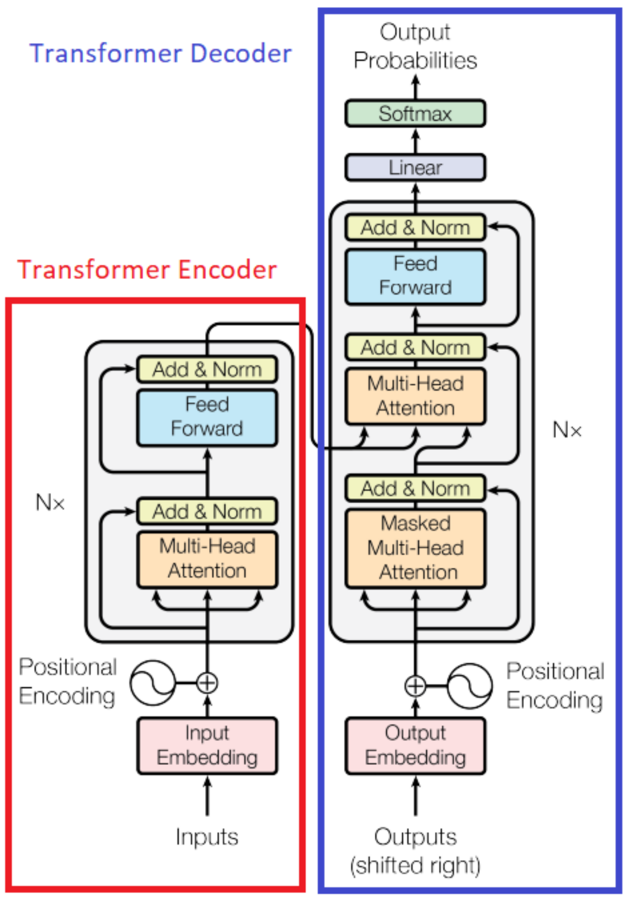


Figure 1: The Transformer - model architecture.

Figure 3: Transformer Architecture [VSP+17]

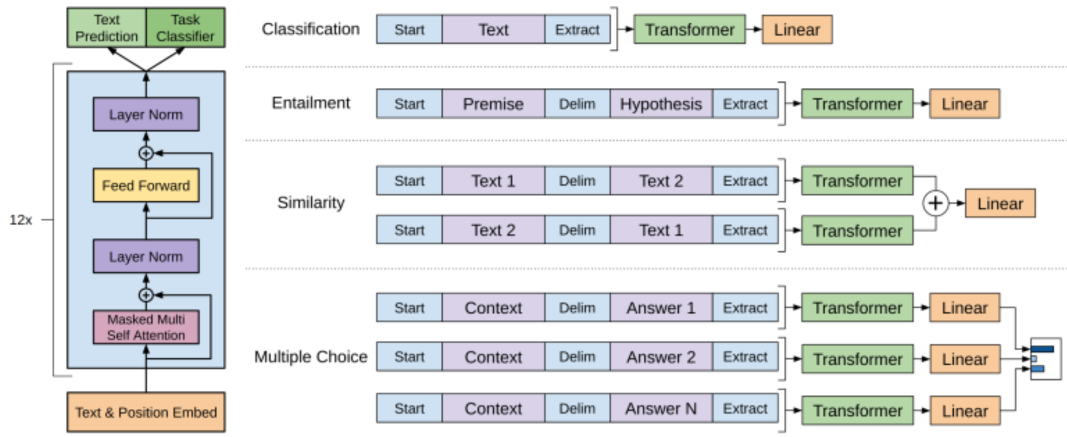


Figure 1: **(left)** Transformer architecture and training objectives used in this work. **(right)** Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

Figure 4: GPT Training Objectives [RNSS18]

where $U = (u_{-k}, ..., u_{-1})$ is the context vector of tokens, n is the number of layers, W_e is the token embedding matrix, and W_p is the position embedding matrix.

Supervised fine-tuning uses labeled training data to further optimize the model for certain tasks, including classification, semantic similarity, textual entailment and question answering. Given a labeled dataset $\mathcal{C}$ of inputs $\{x^1, ..., x^m\}$ and targets y, the model is trained to maximize the likelihood objective:

$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y|x^1, ..., x^m) \tag{23}$$

where

$$P(y|x^1, ..., x^m) = \text{softmax}(h_l^m W_y) \tag{24}$$

h_l^m is the last output of the final transformer block and W_y are learned parameters.

During fine-tuning, the authors continued to use the masked language modeling to improve generalization and accelerate convergence. Hence, the final fine-tuning objective was

$$L_3(\mathcal{C}) = L_2\mathcal{C} + \lambda L_1(\mathcal{C}) \tag{25}$$

with a tuning parameter λ that trades off between the two objectives of performance on the fine-tuning task and generalization.

6 Language Models are Unsupervised Multitask Learners [RWC+18]

[RWC+18] is the paper from OpenAI that introduced GPT-2. The principle finding of this paper was that by scaling up both model size from approximately 100M parameters to 1.5B parameters and using a significantly larger pre-training dataset (WebText - based on Common Crawl data with outbound Redit links), a Transformer-decoder type model could achieve state-of-the-art performance on many tasks without the need to fine-tune.

7 Language models are few-shot learners [BMR+20]

[BMR+20] is the paper from OpenAI that introduced GPT-3. GPT-3 is again a transformer decoder-only model pre-trained on a curated version of the Common Crawl dataset. Again, OpenAI significantly scaled up the size of the model - the largest GPT-3 model has 175B parameters. Like GPT-2, GPT-3 was not fine-tuned for any specific task, but still demonstrated near-SOTA performance across a range of NLP tasks. GPT-3 performed better in few-shot contexts; that is, when examples demonstrating desired output for the task ere provided as part of the prompt.

8 Training language models to follow instructions with human feedback [OWJ+22]

This is the paper from OpenAI that introduced Instruct GPT and the use of Reinforcement Learning from Human Feedback (RLHF). Instruct GPT uses a pre-trained GPT 3 model and fine-tunes it in three steps:

1. Given a set of prompts and desired responses, the model is fine-tuned on these examples in a supervised manor (SFT).
2. Given a prompt, several model outputs are sampled and a human labeler ranks them from best to worst. A dataset generated in this manor is then used to train a reward model (RM) (also a GPT 3 model).
3. Given a prompt, the model from 1 generates an output, the reward model calculates a reward and then the model is updated using reinforcement learning via proximal policy optimization.

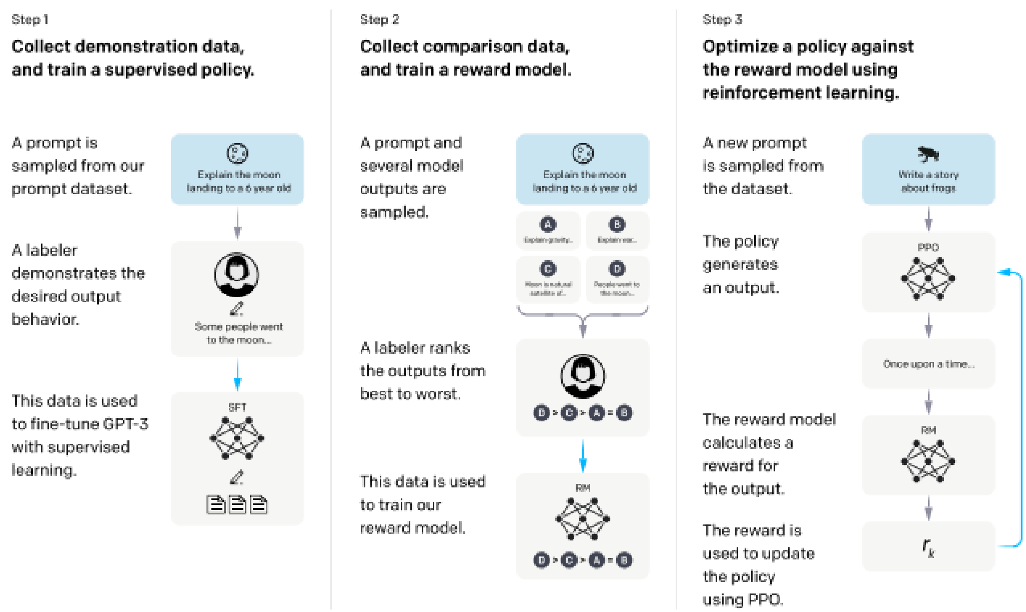


Figure 2: A diagram illustrating the three steps of our method: (1) supervised fine-tuning (SFT), (2) reward model (RM) training, and (3) reinforcement learning via proximal policy optimization (PPO) on this reward model. Blue arrows indicate that this data is used to train one of our models. In Step 2, boxes A-D are samples from our models that get ranked by labelers. See Section 3 for more details on our method.

Figure 5: Instruct GPT Fine-Tuning [OWJ+22]

9 Encoder only transformers

9.1 BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

[DCLT19] was the first paper to introduce BERT models - transformer encoder only models that can be fine-tuned for a variety of tasks. BERT is pretrained using two methods:

1. Masked Language Modeling (MLM)
2. Next Sentence Prediction (NSP)

9.2 RoBERTa

[LOG$^+$19] is a paper from the University of Washington and Facebook that improved the performance of BERT using several modifications, most importantly:

1. Pretraining the model on much more data for much longer
2. Dropping Next Sentence Prediction (NSP) as an objective
3. Dynamically masking tokens instead of masking the same tokens in each sequence for every training epoch

9.3 BART

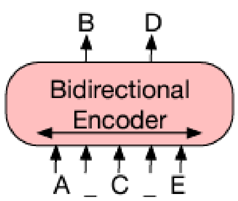
[LLG$^+$19] introduced BART and showed that model performance could be improved by masking spans of text in the transformer encoder. See Figure 6.

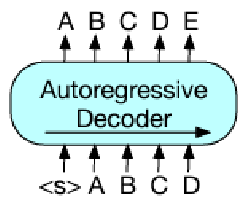
10 Scaling Transformers to Longer Sequences

One limitation of the original transformers is that their computational complexity grows quadratically in the length of the input sequence; that is, they have computational complexity of $O(n^2)$, where n is the sequence length. There have been several approaches developed to reduce this complexity.

10.1 Transformer-XL [DYY$^+$19]

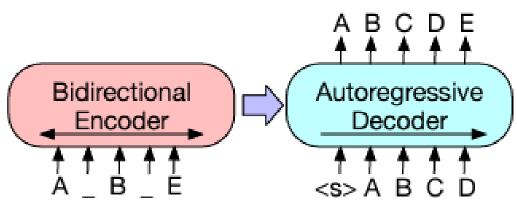
One crude option to reduce computational complexity during training is to split text into segments of length L and train a model on the individual segments, ignoring all contextual information from previous segments. This reduces complexity during training but causes contextual fragmentation as no information flows across segments. This vanilla model is shown in 7. At each time step during evaluation, the model processes a text segment of length L, the last output position is recorded and then the context window is shifted to





(a) BERT: Random tokens are replaced with masks, and the document is encoded bidirectionally. Missing tokens are predicted independently, so BERT cannot easily be used for generation.

(b) GPT: Tokens are predicted auto-regressively, meaning GPT can be used for generation. However words can only condition on leftward context, so it cannot learn bidirectional interactions.



(c) BART: Inputs to the encoder need not be aligned with decoder outputs, allowing arbitary noise transformations. Here, a document has been corrupted by replacing spans of text with mask symbols. The corrupted document (left) is encoded with a bidirectional model, and then the likelihood of the original document (right) is calculated with an autoregressive decoder. For fine-tuning, an uncorrupted document is input to both the encoder and decoder, and we use representations from the final hidden state of the decoder.

Figure 1: A schematic comparison of BART with BERT (Devlin et al., 2019) and GPT (Radford et al., 2018).

Figure 6: BART [LLG+19]

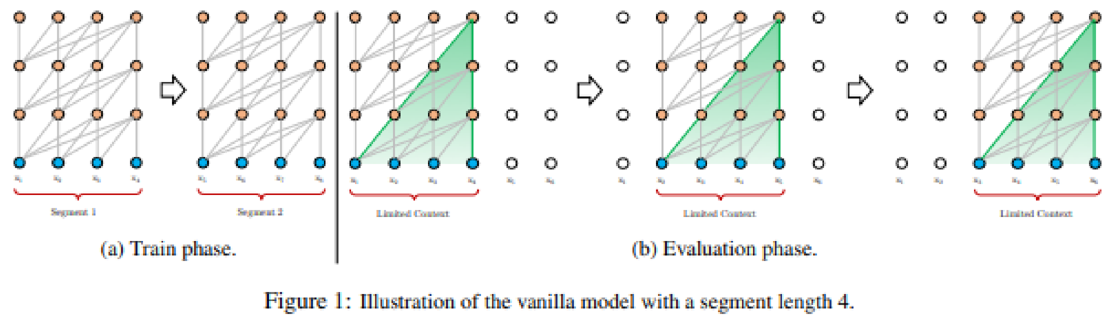


Figure 7: Transformer with Fixed Context Window [DYY+19]

the right by one step and the process repeated. By shifting only a single time step, each prediction is able to use the context of the last L positions, alleviating the contextual fragmentation in training, but reintroducing computational complexity.

In order to address the issue of contextual fragmentation, [DYY+19] introduce a recurrence mechanism. With two consecutive segments of length L, $s_\tau = [x_{\tau,1}, ..., x_{\tau,L}]$ and $s_{\tau+1} = [x_{\tau+1,1}, ..., x_{\tau+1,L}]$, let the n-th layer hidden state sequence produced by the τ-th segment s_τ be $h_\tau^n \in \mathbb{R}^{L \times d}$, where d is the hidden dimension. Then

$$
\begin{aligned}
\tilde{h}_{\tau+1}^{n-1} &= \left[SG(h_\tau^{n-1}) \circ h_{\tau+1}^{n-1}\right] \\
q_{\tau+1}^n, k_{\tau+1}^n, v_{\tau+1}^n &= h_{\tau+1}^{n-1} W_q', \tilde{h}_{\tau+1}^{n-1} W_k', \tilde{h}_{\tau+1}^{n-1} W_v' \\
h_{\tau+1}^n &= \texttt{transformer\_block}(q_{\tau+1}^n, k_{\tau+1}^n, v_{\tau+1}^n)
\end{aligned}
\tag{26}
$$

where SG is the stop-gradient function and $\circ$ denotes concatenation of vectors. Compared to the vanilla transformer, the keys and values in TransformerXL are constructed with an extended context, creating what is essentially a segment-level recurrence in hidden states.

10.2 Generating Long Sequences with Sparse Transformers

In [CGRS19], the authors scale transformers to longer sequences by factorizing the self-attention mechanism. Let $S = \{S_1, ..., S_L\}$, where $S_i \subseteq \{1, ..., L\}$ $\forall i \leq L$. Then

$$\text{Attend}(X, S) = (a(X_{\cdot i}, S_i))_{i \in \{1,...,L\}} \tag{27}$$

$$a(X_{\cdot i}, S_i) = \text{softmax}\left(\frac{(W_q X_{\cdot i}) K_{S_i}'}{\sqrt{d}}\right) V_{S_i} \tag{28}$$

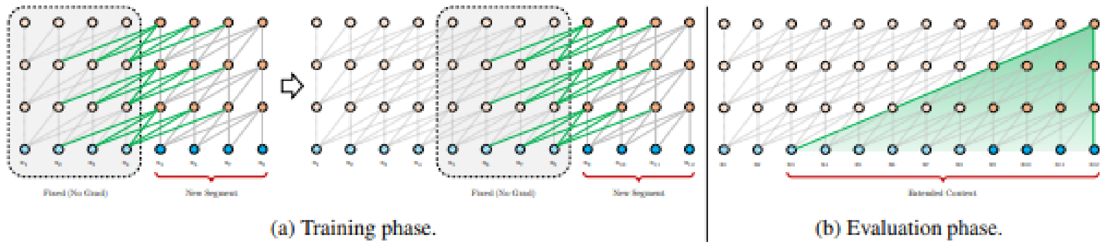


Figure 2: Illustration of the Transformer-XL model with a segment length 4.

Figure 8: TransformerXL [DYY+19]

$$K_{S_i} = (W_k X_{\cdot j})_{j \in S_i} \qquad V_{S_i} = (W_v X_{\cdot j})_{j \in S_i} \tag{29}$$

For full self-attention in an autoregressive model, $S_i = \{j : j \leq i\}$ so that each element attends to all previous positions and its own position.

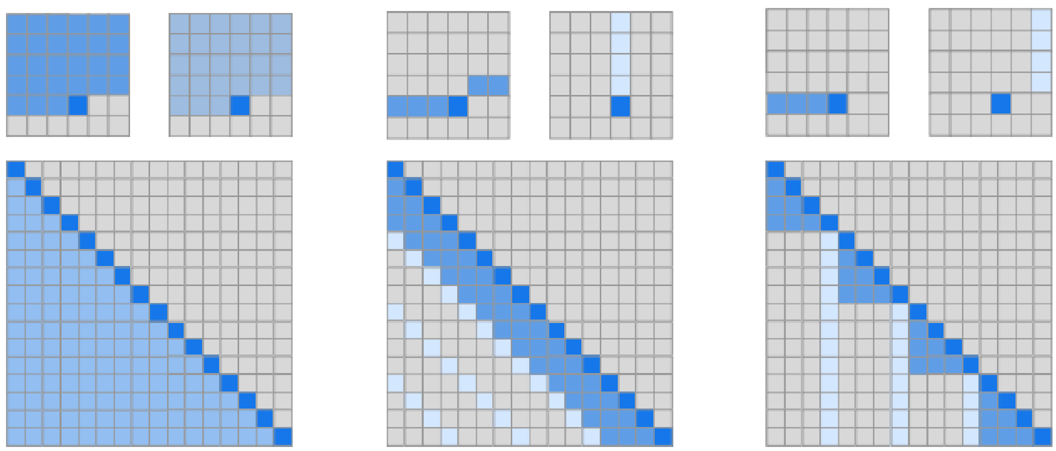
On the other hand, factorized self-attention uses p separate attention heads, where the mth head defines a subset of the indices $A_i^{(m)} \subset \{j : j \leq i\}$. In particular, the authors consider two factorizations:

- Strided Attention sets $A_i^{(1)} = \{t, t+1, ..., i\}$ for $t = \max(0, i - l)$ and $A_i^{(2)} = \{j : (i - j) \text{mod} l = 0\}$.
- Fixed Attention sets $A_i^{(1)} = \{j : (\lfloor j/l \rfloor = \lfloor i/l \rfloor)\}$ and $A_i^{(2)} = \{j : j \text{mod} l \in \{t, t+1, ..., l\}, t = l - c\}$.

where l is the stride length and c is a hyperparameter. Strided attention performs best for data with structure that aligns with the stride, like images. Fixed attention performs well on data without a periodic structure, such as text.

10.3 Longformer [BPC20]

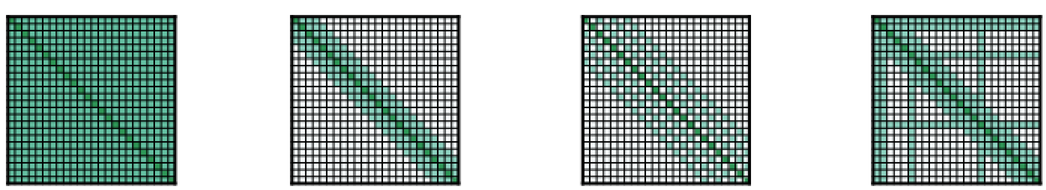
Longformer is a paper from AllenAI that introduced the Longformer model. Similar to sparse transformers, Longformer alters the attention pattern so that not all positions attend to all other positions. Longformer reduces the computational complexity $O(n^2)$ of full attention to $O(n \times w)$ by using a sliding window of size w. The paper also dilates the windows with gaps of size dilation d so that the final receptive field is $l \times d \times w$.



(a) Transformer (b) Sparse Transformer (strided) (c) Sparse Transformer (fixed)

Figure 3. Two 2d factorized attention schemes we evaluated in comparison to the full attention of a standard Transformer (a). The top row indicates, for an example 6x6 image, which positions two attention heads receive as input when computing a given output. The bottom row shows the connectivity matrix (not to scale) between all such outputs (rows) and inputs (columns). Sparsity in the connectivity matrix can lead to significantly faster computation. In (b) and (c), full connectivity between elements is preserved when the two heads are computed sequentially. We tested whether such factorizations could match in performance the rich connectivity patterns of Figure 2.

Figure 9: Sparse Transformer Attention [CGRS19]



(a) Full n^2 attention (b) Sliding window attention (c) Dilated sliding window (d) Global+sliding window

Figure 2: Comparing the full self-attention pattern and the configuration of attention patterns in our Longformer.

Figure 10: Longformer Attention [BPC20]

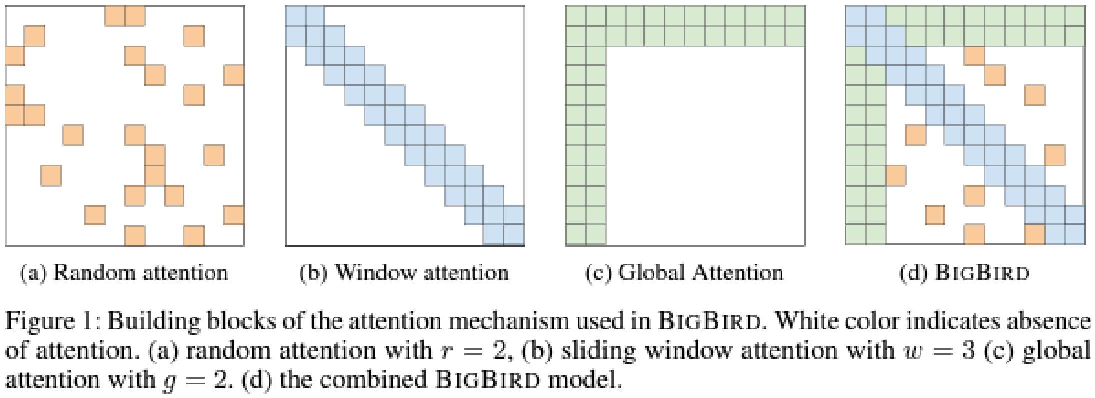


Figure 11: Big Bird Attention [ZGD+21]

10.4 Big Bird: Transformers for Longer Sequences

11 Distilling

11.1 DistilBert, a distilled version of BERT: smaller, faster, cheaper and lighter

[SDCW20] is the paper from Huggingface that introduced model distillation, whereby a large teacher model is used to train a smaller student model. The authors were able to reduce the size of a BERT model by 40% while retaining 97% of its language understanding capabilities and being 60% faster.

11.2 Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes

[HLY+23] is a paper from U. Washington and Google that provided a technique for distilling knowledge from LLMs by prompting them for rationales for their outputs and then training a smaller model on the rationales. See Figure 12.

12 Improving Fine-tuning

12.1 LoRA: Low-Rank Adaptation of Large Language Models [HSW+21b]

LoRA is a method for fine-tuning large, pre-trained transformers with limited fine-tuning data and/or compute. LoRA freezes pre-trained model weights and injects trainable rank decomposition matrices into each layer of the transformer architecture, significantly reducing the number of trainable parameters for downstream tasks.

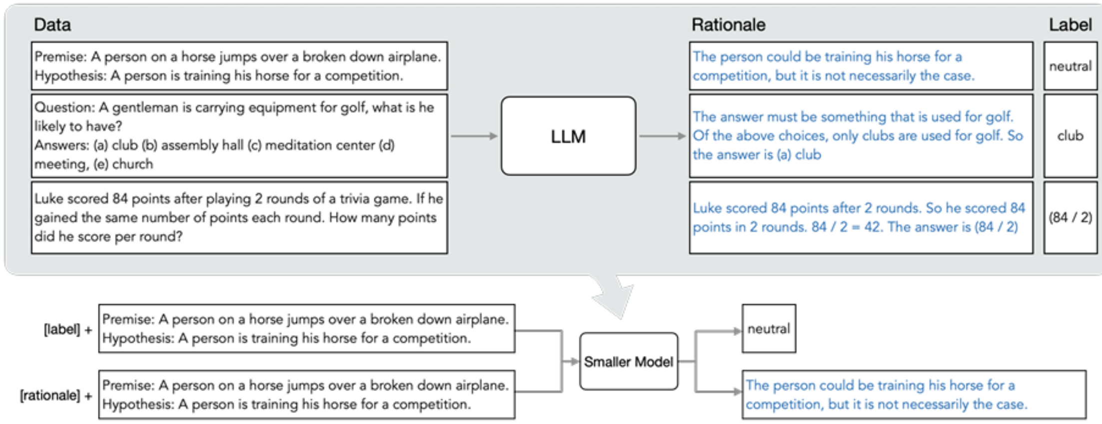


Figure 2: Overview on Distilling step-by-step. We first utilize CoT prompting to extract rationales from an LLM (Section 3.1). We then use the generated rationales to train small task-specific models within a multi-task learning framework where we prepend task prefixes to the input examples and train the model to output differently based on the given task prefix (Section 3.2).

Figure 12: Distilling Step-by-Step [HLY+23]

Given a pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$, during fine-tuning we replace W_0 with frozen$(W_0) + BA$ where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ are trainable parameters, rank$(r) \ll \min(d,k)$, A has a random Gaussian initialization and B is initialized to zero.

12.2 QLoRA: Efficient Finetuning of Quantized LLMs

[DPHZ23] is the paper that introduced the Guanaco family of models, which combined LoRA with a quantization method for efficiently finetuning LLMs with limited compute and memory. The authors of the paper were able to finetune a LLaMA 65B parameter model, which usually would require >780GB of GPU memory, on a single GPU with <48GB without degrading runtime or decreasing performance.

To quantize a 32-bit floating point (FP32) into an Int8 tensor with range $[-127, 127]$:

$$X^{\mathrm{Int8}} = \mathrm{round}\left(\frac{127}{\mathrm{absmax}\left(X^{\mathrm{FP32}}\right)} X^{\mathrm{FP32}}\right) = \mathrm{round}\left(c^{\mathrm{FP32}} \cdot X^{\mathrm{FP32}}\right) \tag{30}$$

where c^{FP32} is a quantization constant. To dequantize, calculate:

$$\mathrm{dequant}\left(c^{\mathrm{FP32}}, X^{\mathrm{Int8}}\right) = \frac{X^{\mathrm{Int8}}}{c^{\mathrm{FP32}}} = X^{\mathrm{FP32}} \tag{31}$$

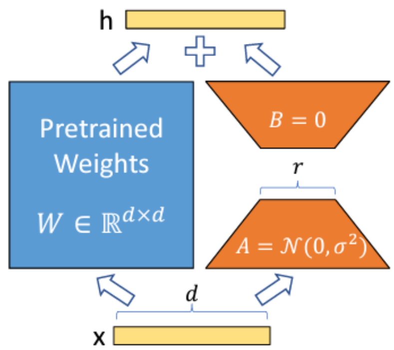


Figure 13: LoRA Fine-tuning Strategy [HSW+21a]

One issue with quantization is that outliers in the input tensor can produce quantization bins that are not used well, that is the distribution of numbers quantized into the bins is very far from uniform. Two ways to prevent this are to:

1. chunk the input tensor into B blocks which are independently quantized, producing B quantization constants $\{c_i : 1 \leq i \leq B\}$
2. quantize into the quantiles of the input tensor rather than into linearly-spaced bins.

[DPHZ23] use 4-bit normal float quantile quantization. To perform a k-bit normal float quantile quantization:

1. Calculate
$$q_i = \begin{cases} \frac{i}{2^k} & 1 \leq i \leq 2^{k-1} \\ \frac{1}{2} + \frac{1}{2}\frac{i-2^{k-1}}{2^{k-1}+1} & 2^{k-1} < i \leq 2^k \end{cases} \tag{32}$$
2. Calculate $x_i = Q(q_i) \quad \forall i$, where Q is the quantile function of the standard Normal distribution.
3. Calculate
$$z_i = \begin{cases} \frac{x_i}{|x_1|} & 1 \leq i \leq 2^{k-1} \\ \frac{x_i}{|x_{2^k}|} & 2^{k-1} < i \leq 2^k \end{cases} \tag{33}$$

[DPHZ23] take quantization one step further with double quantization whereby they quantize the quantization constants. In particular they take the quantization constants from

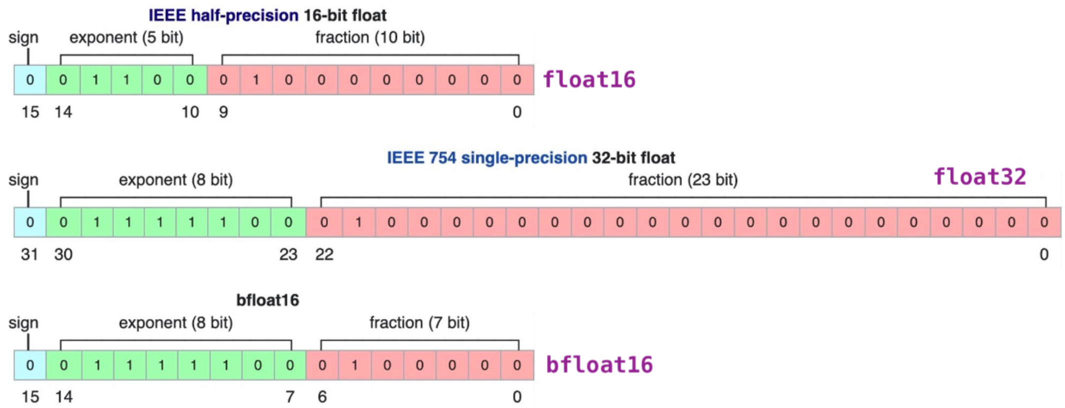


Figure 14: bfloat16 [bfl]

first stage 4-bit normal float quantile quantization and quantize these. The output of a singular linear layer is now calculated as:

$$Y^{BF16} = X^{BF16}\text{doubleDequant}(c_1^{FP32}, c_2^{FP8}, W^{NF4}) + X^{BF16}L_1^{BF16}L_2^{BF16}, \tag{34}$$

where doubleDequant$(\cdot)$ is defined as:

$$\text{doubleDequant}(c_1^{FP32}, c_2^{FP8}, W^{NF4}) = \text{dequant}\left(\text{dequant}\left(c_1^{FP32}, c_2^{FP8}\right), W^{NF4}\right) \tag{35}$$

Data types used in QLoRA are:

1. Pre-trained model weights
   (a) N 4-bit Normal Float to store the double quantized values
   (b) $\frac{N}{B_1}$ 8-bit FP8 to store the first quantization constants
   (c) $\frac{N}{B_1 B_2}$ 32-bit FP32 to store the second quantization constants
2. Optimizer state - 32-bits per LoRA adapter parameter in 2 bfloat16
3. LoRA weights - 16-bits per LoRA adapter parameter in bfloat16
4. Activation gradients - 16-bit bfloat16
5. LoRA weight gradients in bfloat16

13 Relative Position Embeddings

Relative position embeddings were introduced by [SUV18].

Given relative position embedding matrix $E^r \in \mathbb{R}^{L \times d_{model}}$ and matrix $X \in \mathbb{R}^{L \times d_{model}}$

Algorithm 1: Relative Position Embedding of [HVU+18]

Input: Relative embedding matrix $E^r \in \mathbb{R}^{L \times d_{model}}$
Matrix $X \in \mathbb{R}^{L \times d_{model}}$

Output: $D \in \mathbb{R}^{L \times L}$

$A \leftarrow X E^{rT} \in \mathbb{R}^{L \times L}$

$M \leftarrow \begin{cases} m_{i,j} = 1 & i \leq L, j \leq L, i \geq j \\ m_{i,j} = 0 & i \leq L, j \leq L, i < j \end{cases} \in \mathbb{R}^{L \times L}$

$A \leftarrow A \odot M$ // element-wise product

$B \leftarrow \begin{cases} b_{i,j} = 0 & j = 1, i \leq L \\ b_{i,j} = a_{i,j-1} & i \leq L, 1 < j \leq L+1 \end{cases} \in \mathbb{R}^{L \times L+1}$

$V \leftarrow \mathrm{vec}(B^T)$

$C \leftarrow \{c_{ij} = V_{(i-1)L+j} \mid i \leq L, j \leq L\} \in \mathbb{R}^{L+1 \times L}$

$D \leftarrow \{d_{ij} = c_{i+1,j} \mid i \leq L, j \leq L\}$

return D

14 Prompt Design and Tuning

14.1 Prefix Tuning

[LL21] is a paper from researchers at Stanford that introduced prefix tuning. Prefix tuning freezes the parameters of a pretrained transformer but then prefixes each layer with a set of tuneable parameters.

In particular, given inputs x and outputs y, let

$$z = \begin{cases} [PREFIX; x; y] & \text{Encoder} - \text{only transformer} \\ [PREFIX; x; PREFIX'; y] & \text{Encoder} - \text{Decoder transformer} \end{cases} \tag{36}$$

Let $\mathtt{P}$ denote the indices of the prefixes. For a pretrained model with L layers and model dimension d and prefix length $|\mathtt{P}|$, we have a matrix of trainable prefix parameters $P_\theta \in \mathbb{R}^{|\mathtt{P}|} \times \mathbb{R}^d$. The activations at time step t for layer l are then

$$h_t^{(l)} = \begin{cases} P_\theta\,[t,:] & t \in \mathtt{P} \\ \mathrm{LM}_\phi(z_t, h_{<t})^{(l)} & \text{otherwise} \end{cases} \tag{37}$$

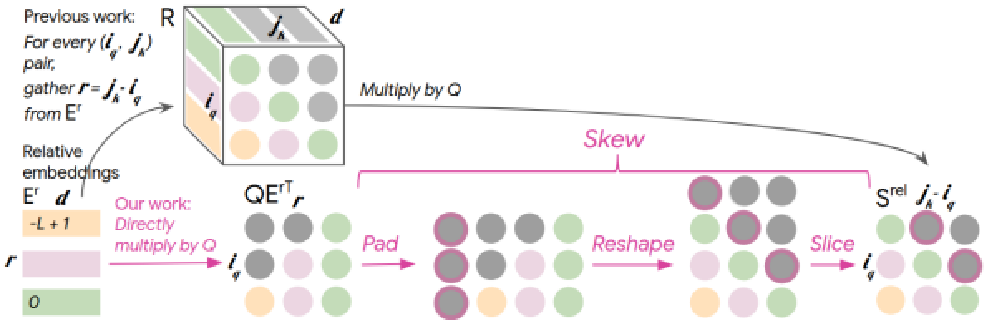


Figure 1: Relative global attention: the bottom row describes our memory-efficient "skewing" algorithm, which does not require instantiating R (top row, which is $O(L^2D)$). Gray indicates masked or padded positions. Each color corresponds to a different relative distance.

Figure 15: Relative Global Attention [HVU+18]

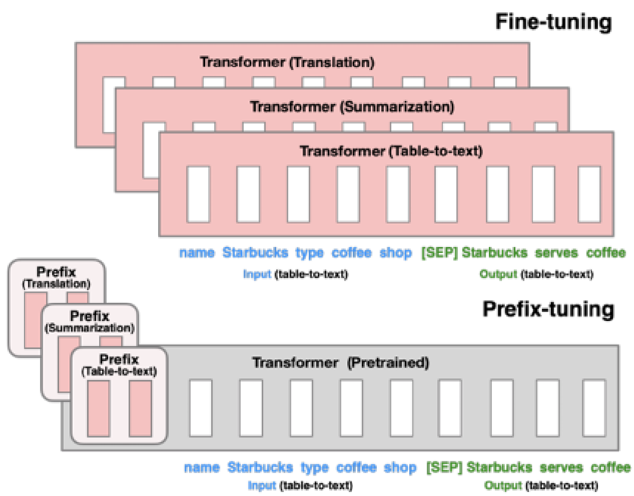


Figure 1: Fine-tuning (top) updates all Transformer parameters (the red Transformer box) and requires storing a full model copy for each task. We propose prefix-tuning (bottom), which freezes the Transformer parameters and only optimizes the prefix (the red prefix blocks). Consequently, we only need to store the prefix for each task, making prefix-tuning modular and space-efficient. Note that each vertical block denote transformer activations at one time step.

Figure 16: Prefix-tuning [LL21]

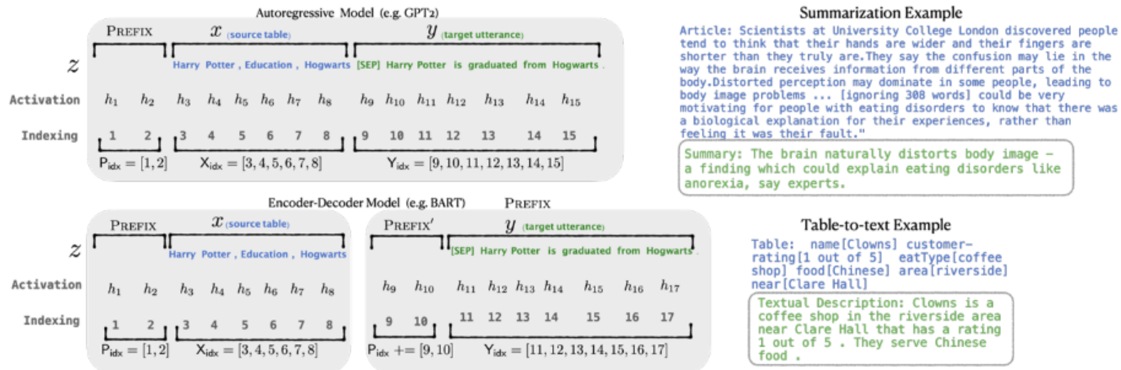


Figure 2: An annotated example of prefix-tuning using an autoregressive LM (top) and an encoder-decoder model (bottom). The prefix activations $\forall i \in \mathsf{P}_{\mathsf{idx}}, h_i$ are drawn from a trainable matrix P_θ. The remaining activations are computed by the Transformer.

Figure 17: Prefix-tuning Example [LL21]

14.2 Prompt Tuning

[LARC21] is a paper from Google Research that introduced prompt tuning. Prompt tuning is a simplification of prefix tuning in which a set of tuneable parameters are prepended to the token embeddings of just the inputs instead of the activations of each layer in the transformer. These modified token embeddings are then propogated through the model as before.

15 Mixture-of-Experts

15.1 Sparsely-Gated Mixture-of-Experts Layer

[SMM+17] introduced the Mixture-of-Experts layer

The mixture-of-experts layer uses a gating function $G(x)$ where:

$$G(x) = Softmax(KeepTopK(H(x), k)) \tag{38}$$

$$H(x)_i = (x \cdot W_g)_i + StandardNormal() \cdot Softplus((x \cdot W_{noise})_i) \tag{39}$$

$$KeepTopK(v, k)_i = \begin{cases} v_i & v_i \text{ in top } k \text{ elements of } v \\ -\infty & \text{otherwise} \end{cases} \tag{40}$$

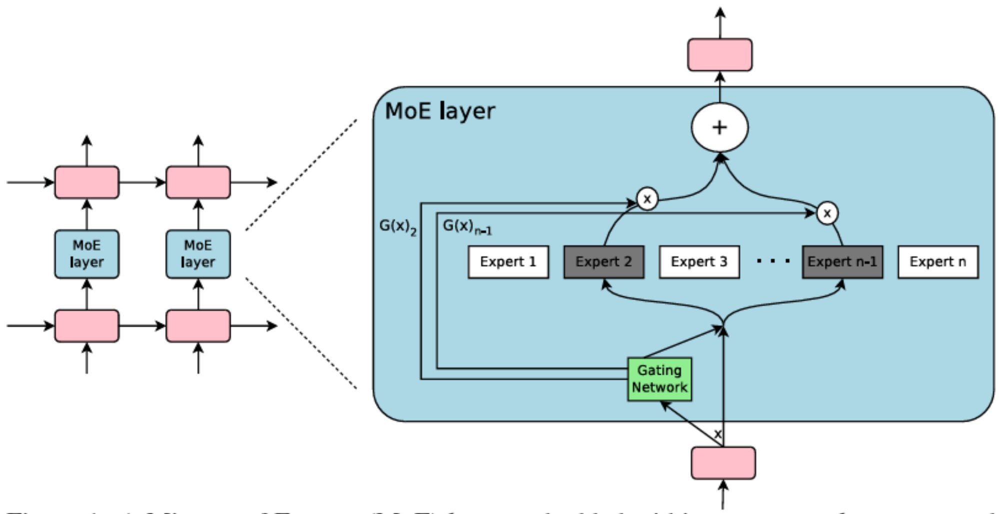


Figure 1: A Mixture of Experts (MoE) layer embedded within a recurrent language model. In this case, the sparse gating function selects two experts to perform computations. Their outputs are modulated by the outputs of the gating network.

Figure 18: Mixture-of-Experts Layer [SMM+17]

15.2 Switch Transformer

[FZS22] introduced the Switch Transformer architecture

16 Direct Preference Optimization

16.1 Direct Preference Optimization

[RSM+23] is the paper from Stanford that introduced Direct Preference Optimization as an alternative to RLHF that is ”stable, performant, and computationally lightweight,” and can ”fine-tune LMs to align with human preferences as well as or better than existing methods.” The key insight of the paper is that the objective function in training a reward model in RLHF can be reparameterized into a form that does not contain the reward function itself, but only the optimal policy function (with respect to the implicit reward function). In other words, there is a one-to-one correspondence between reward functions (of a certain kind) and the corresponding optimal policy function and the relationship is easily invertible in a closed form.

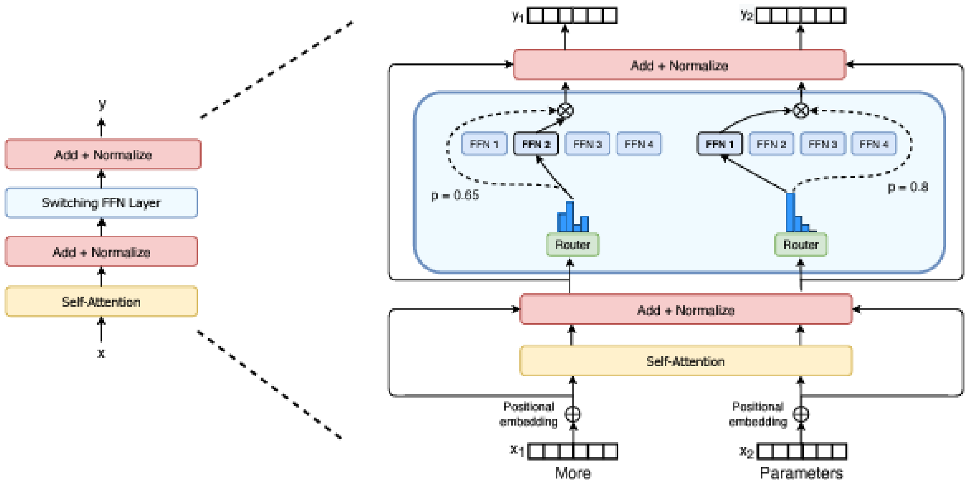


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = "More" and x_2 = "Parameters" below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

Figure 19: Switch Transformer [FZS22]

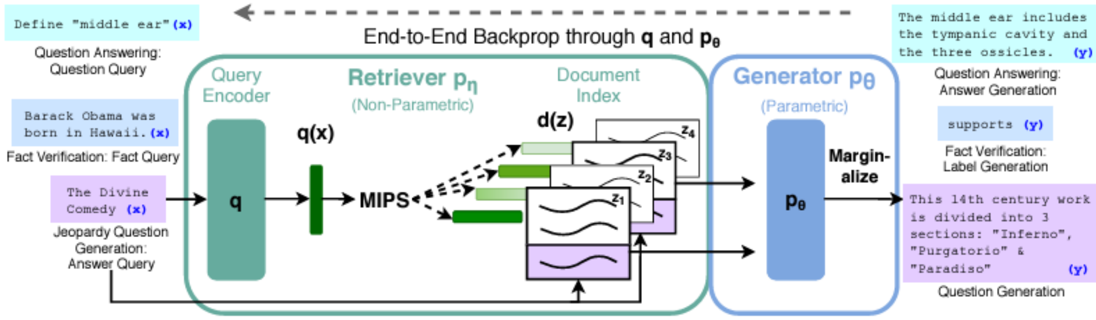


Figure 1: Overview of our approach. We combine a pre-trained retriever (*Query Encoder* + *Document Index*) with a pre-trained seq2seq model (*Generator*) and fine-tune end-to-end. For query x, we use Maximum Inner Product Search (MIPS) to find the top-K documents z_i. For final prediction y, we treat z as a latent variable and marginalize over seq2seq predictions given different documents.

Figure 20: Retrieval-Augmented Generation [LPP$^+$21]

17 Retrieval-Augmented Generation

[LPP$^+$21] is a paper from Facebook Research that introduced RAG.

18 Byte-Pair Encoding

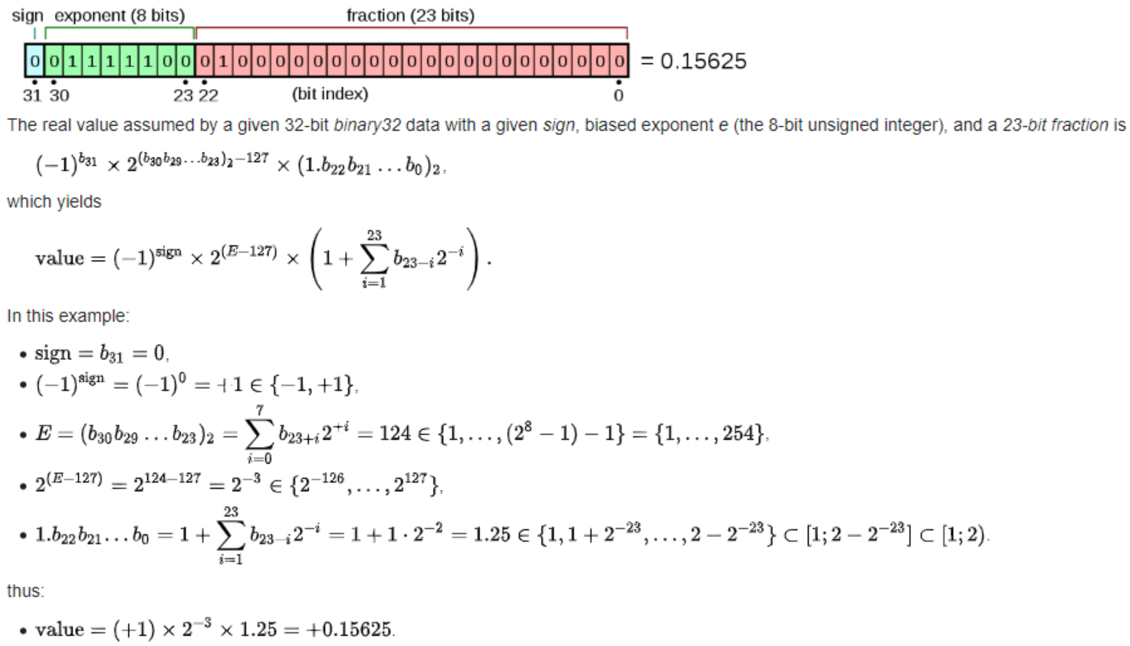
```python
import re, collections

def get_stats(vocab):
    pairs = collections.defaultdict(int)
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols)-1):
            pairs[symbols[i],symbols[i+1]] += freq
    return pairs

def merge_vocab(pair, v_in):
    v_out = {}
    bigram = re.escape(' '.join(pair))
    p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
    for word in v_in:
        w_out = p.sub(''.join(pair), word)
        v_out[w_out] = v_in[word]
    return v_out

vocab = {'l o w </w>' : 5, 'l o w e r </w>' : 2,
         'n e w e s t </w>':6, 'w i d e s t </w>':3}

num_merges = 10

for i in range(num_merges):
    pairs = get_stats(vocab)
    best = max(pairs, key=pairs.get)
    vocab = merge_vocab(best, vocab)
    print(best)
```

Listing 1: Byte-Pair Encoding

19 Numeric Types and Precision

More papers to cover:

1. Roformer
2. FLAN T5 - Fine-tuning Language Net - instruction tuning
3. Roberta
4. Albert
5. Switch Transformer
6. GLaM



The real value assumed by a given 32-bit *binary32* data with a given *sign*, biased exponent e (the 8-bit unsigned integer), and a *23-bit fraction* is

$$(-1)^{b_{31}} \times 2^{(b_{30}b_{29}\dots b_{23})_2-127} \times (1.b_{22}b_{21}\dots b_0)_2,$$

which yields

$$\text{value} = (-1)^{\text{sign}} \times 2^{(E-127)} \times \left(1 + \sum_{i=1}^{23} b_{23-i}2^{-i}\right).$$

In this example:

- $\text{sign} = b_{31} = 0$,
- $(-1)^{\text{sign}} = (-1)^0 = +1 \in \{-1, +1\}$,
- $E = (b_{30}b_{29}\dots b_{23})_2 = \sum_{i=0}^{7} b_{23+i}2^{+i} = 124 \in \{1, \dots, (2^8-1)-1\} = \{1, \dots, 254\}$,
- $2^{(E-127)} = 2^{124-127} = 2^{-3} \in \{2^{-126}, \dots, 2^{127}\}$,
- $1.b_{22}b_{21}\dots b_0 = 1 + \sum_{i=1}^{23} b_{23-i}2^{-i} = 1 + 1\cdot 2^{-2} = 1.25 \in \{1, 1+2^{-23}, \dots, 2-2^{-23}\} \subset [1; 2-2^{-23}] \subset [1; 2)$.

thus:

- $\text{value} = (+1) \times 2^{-3} \times 1.25 = +0.15625$.

Figure 21: float32 [flo]

7. QLoRA
8. Unigrams
9. SentencePiece
10. Mixture of Experts
11. Chain of Thought Prompting
12. Contrastive Pre-training - this may belong in a separate document
13. Improving alignment of dialogue agents via targeted human judgements

References

[bfl] To bfloat or not to bfloat? that is the question! https://www.cerebras.net/machine-learning/to-bfloat-or-not-to-bfloat-that-is-the-question/. Accessed: 2023-09-21.

[BMR+20] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.

[BPC20] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020.

[CGRS19] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019.

[DCLT19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.

[DPHZ23] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023.

[DYY+19] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context, 2019.

[flo] Single-precision floating-point format. `https://en.wikipedia.org/wiki/Single-precision_floating-point_format`. Accessed: 2023-10-02.

[FZS22] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2022.

[HLY+23] Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes, 2023.

[HSW+21a] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *CoRR*, abs/2106.09685, 2021.

[HSW+21b] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.

[HVU+18] Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Ian Simon, Curtis Hawthorne, Andrew M. Dai, Matthew D. Hoffman, Monica Dinculescu, and Douglas Eck. Music transformer, 2018.

[LARC21] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning, 2021.

[LL21] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation, 2021.

[LLG+19] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension, 2019.

[LOG+19] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019.

[LPP+21] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.

[OWJ+22] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022.

[RNSS18] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. 2018.

[RSM+23] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2023.

[RWC+18] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2018.

[Sch17] Natalie Schluter. The limits of automatic summarisation according to ROUGE. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pages 41–45, Valencia, Spain, April 2017. Association for Computational Linguistics.

[SDCW20] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter, 2020.

[SMM+17] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017.

[SUV18] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations, 2018.

[VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017.

[ZGD+21] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences, 2021.